

DESIGN CASE STUDY

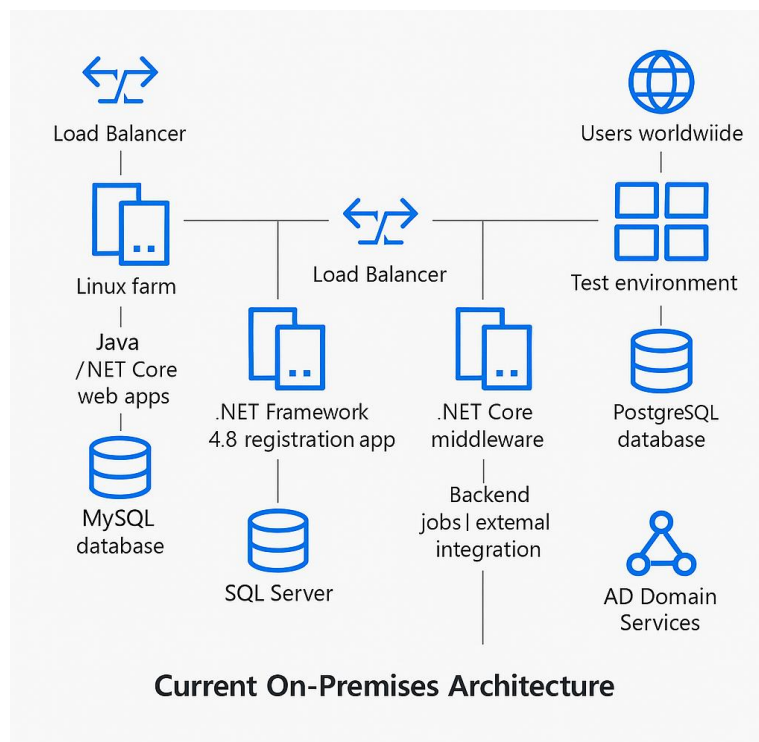
Introduction to Cloud Computing

Muhammad Hamza (21L-5636)

Muhammad Hassan Khalid (21L-5692)

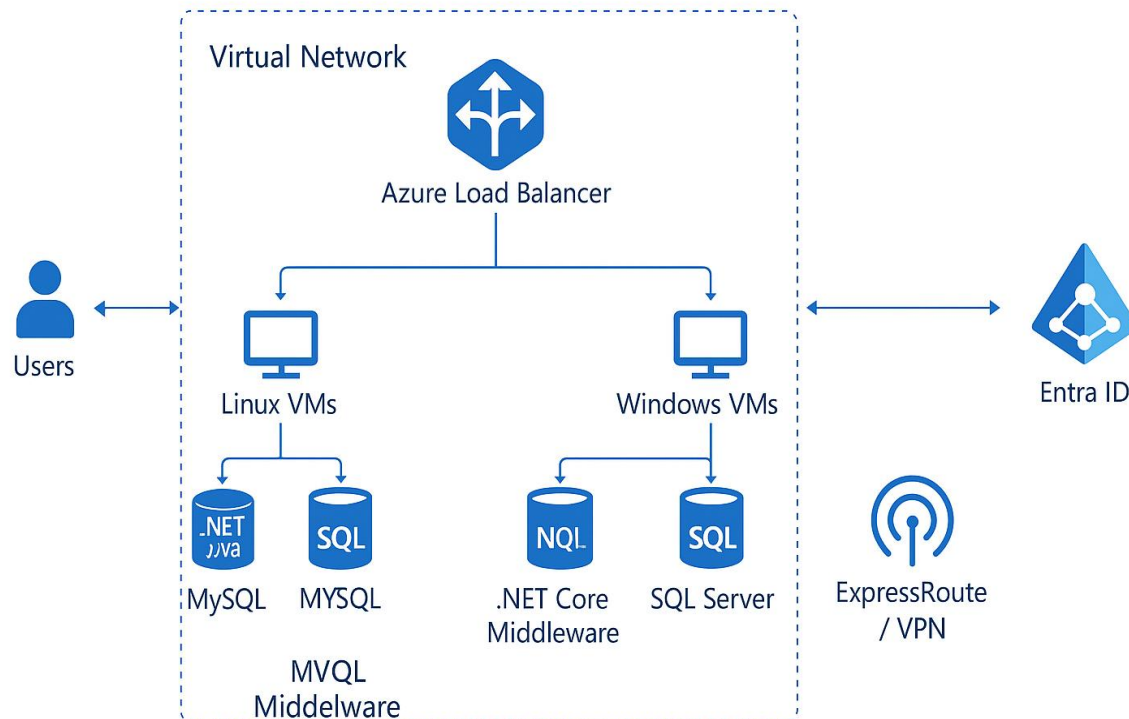
Current On-Premises Architecture

Contoso's current environment comprises multiple on-prem server farms and databases. A Linux farm (6 servers behind a load balancer) hosts Java and .NET Core web apps, including the main website (which also serves large PDF catalogs) connected to a MySQL database. A Windows Server farm (behind another load balancer) runs a .NET Framework 4.8 registration app using an on-premises PostgreSQL database. A third load-balanced .NET Core middleware farm handles backend jobs (email, jobs, external integration). There is also an on-premises SQL Server supporting various apps, with scheduled jobs. A separate test environment mirrors all production apps and databases. Student identities live in the on-prem AD Domain Services. Users worldwide (mainly in North America/Europe) download the large catalog PDFs, resulting in slow download speeds. Security requires that all databases be isolated in separate network segments when moved to the cloud.



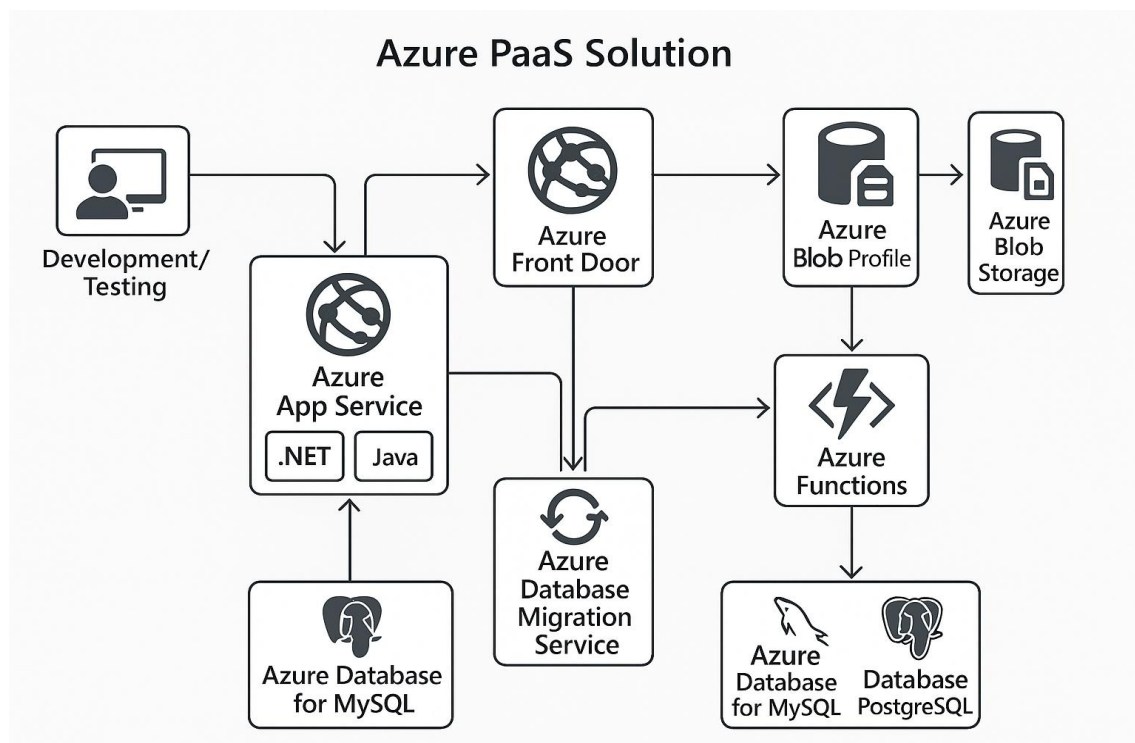
Azure IaaS Solution (Lift-and-Shift)

In the IaaS design, each on-prem component becomes an Azure VM or VM scale set behind Azure Load Balancers in one or more virtual networks (VNETs). For example, use Linux VMs (or scale sets) behind an Azure Load Balancer for the Java/.NET web farm, and Windows VMs for the .NET registration farm, each pointing to their respective databases. Each database (MySQL, PostgreSQL, and SQL Server) runs on an Azure VM in its own secured subnet/VNet to satisfy the isolation requirement. ExpressRoute or VPN can connect Azure VNETs to any remaining on-premises resources (if needed). All servers run in scale sets with auto-scaling, and Azure Load Balancer distributes traffic, matching the on-prem pattern. To minimize cost, we will right-size VM SKUs based on performance-based sizing (from Azure Migrate assessments) and use Reserved Instances with Azure Hybrid Benefit for Windows licenses. Windows Server AD could be replaced by Azure Active Directory (Entra ID) Domain Services or synchronized with on-prem AD to allow existing credentials, but we leverage Entra ID as the identity provider (see below). This lift-and-shift design reuses compute by using autoscaling and reserved capacity and simply moves servers to Azure VMs.



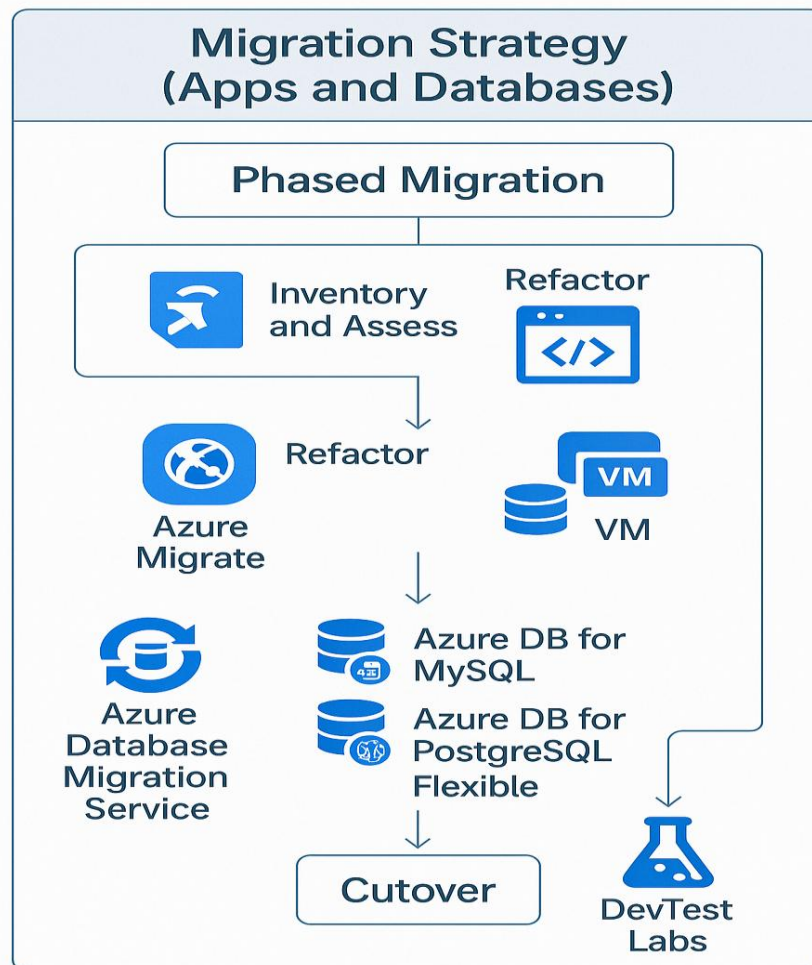
Azure PaaS Solution

In the PaaS design, we replace VMs with managed services wherever possible. The web applications move to **Azure App Service** (one or more Web Apps) running the .NET and Java sites; App Service is a fully managed platform for web apps. The main website and registration apps run in App Services (multi-tenant PaaS), and can share an App Service Plan if capacity allows. The backend MySQL and PostgreSQL databases migrate to **Azure Database for MySQL** and **Azure Database for PostgreSQL (Flexible Server)**, respectively, using Azure Database Migration Service to minimize downtime. For example, Azure DMS can perform an online migration of Postgres with continuous replication to achieve minimal downtime. Server-side middleware jobs become **Azure Functions** (serverless, event-driven code), which automatically scale with demand. Static PDFs and media are stored in **Azure Blob Storage**, and an **Azure CDN/Front Door** profile fronts the storage to cache and accelerate downloads globally. Azure Front Door specifically “enhances delivery of static content from Azure Storage” for secure, scalable global access, addressing the slow catalog downloads. A global Front Door or CDN handles requests in North America, Europe, etc., dramatically improving latency. For development/testing, we provision **Azure DevTest Labs** workspaces. Azure DevTest Labs “makes it easy and fast for enterprises to provide access to Azure resources” and supports creating controlled VM and PaaS environments for testing. Developers and QA can deploy test versions of apps into DevTest Lab VMs or App Services, and then roll changes to production via App Service deployment slots or release pipelines.



Migration Strategy (Apps and Databases)

We will use a phased migration. First, inventory and assess using Azure Migrate to identify dependencies and right-size resources. For each app, we choose the appropriate approach: simple web apps (especially the .NET Core site) can be refactored to App Service (PaaS) for minimal management, whereas complex apps may be lifted to VMs. We migrate databases using Azure Database Migration Service (DMS): for MySQL, use Azure DB for MySQL with an offline or online migration as needed; for PostgreSQL, use Azure DB for PostgreSQL (Flexible) with DMS, leveraging its online migration for minimal downtime. During cutover, applications will be switched to point to the new Azure databases. We prioritize low-risk workloads first, test in DevTest Labs, then proceed to production. All migrations follow best practices (e.g. backup/restore, validation). By using managed PaaS databases and App Service, we reduce management overhead and improve scalability.

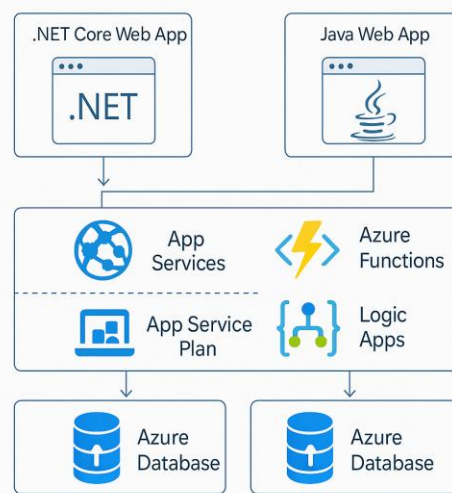


Compute Reuse and Cost Minimization

We optimize compute by using auto-scaling and managed services. VM workloads use Azure VM Scale Sets, enabling horizontal scaling under load and consistent configurations. We will use Azure Migrate's performance-based sizing to right-size VMs. Cost savings come from Azure Reserved Instances (for predictable VMs) and Hybrid Benefit (using existing Windows licenses). Non-production VMs (test labs) run under Dev/Test subscriptions to get reduced pricing. Wherever possible, we "shut off servers when not in use" (DevTest labs support auto-shutdown) to save cost. PaaS services inherently scale on demand (App Service and Functions scale automatically), so compute is only consumed when needed. In summary, we reuse existing workload designs (load balancers, multiple tiers) but replace fixed servers with elastic Azure resources, ensuring we only pay for what we use and reserve capacity for baseline loads.

Azure-Native PaaS Utilization and Reuse

Our architecture prioritizes Azure-native PaaS to reduce management and improve agility. The .NET Core and Java web apps run on App Services (multi-tenant web apps). Any common functionality (e.g. shared libraries or middleware jobs) is implemented via Azure Functions or Logic Apps to avoid running dedicated servers. Azure Functions abstracts the compute and auto-scales ("you can use Azure Functions to run event-driven code without managing infrastructure"). The databases use fully managed Azure Database services instead of VMs. We also plan to reuse PaaS instances: for example, multiple web apps can share an App Service Plan if resource budgets allow, and all databases can be managed by the corresponding Azure DB service for easier updates. This reuse of cloud-native services increases consistency and leverages Azure's scale-out capabilities.



All databases can be managed by corresponding Azure DB service

Test Environments and Production Rollouts

To enable safe testing and smooth releases, we use Azure DevTest Labs and Azure App Service deployment features. Development and QA teams will use DevTest Labs to spin up isolated labs with identical resources (VMs, App Service instances, databases) as production. DevTest Lab policies automate provisioning and cleanup, and Dev/Test pricing cuts costs. For production rollout, we use Azure App Service deployment slots or staging environments. In App Service, we create a staging slot for each web app. New versions are first deployed to the staging slot, smoke-tested, and then swapped into production with no downtime. As noted in reference architectures, **deployment slots** eliminate downtime during updates. This slot swap approach (or blue/green deployment) ensures continuous availability. After testing in DevTest and in a staging slot, changes can be promoted to production seamlessly.

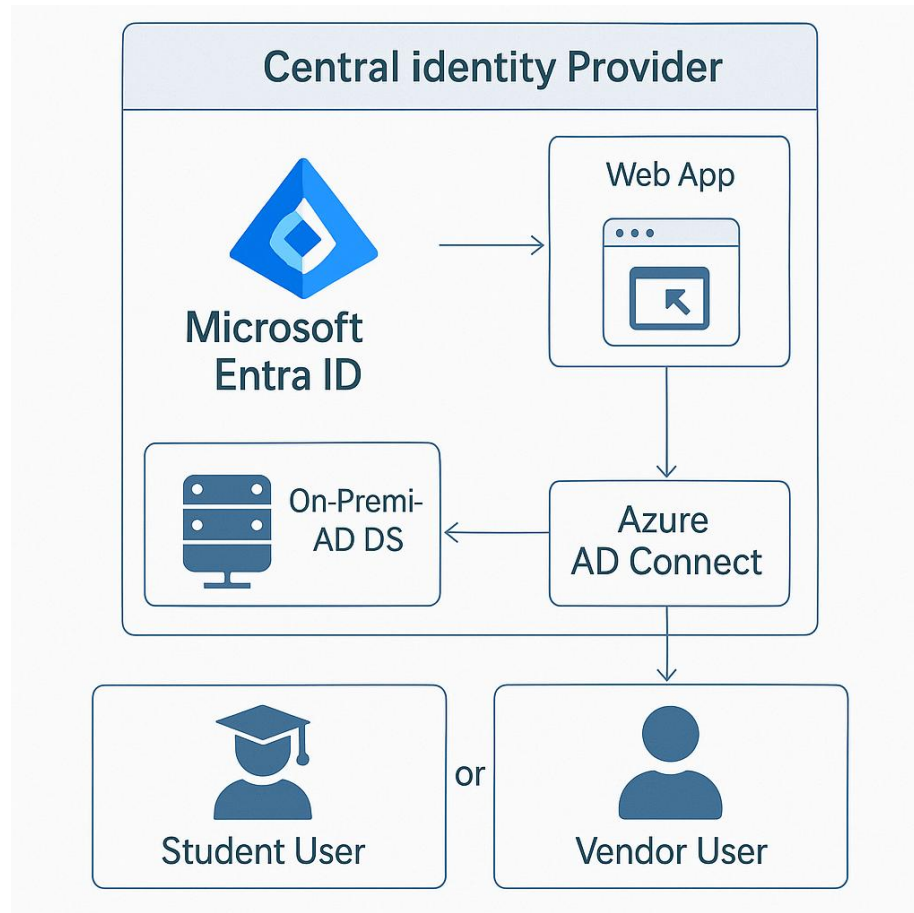
Global Catalog Download Optimization

To accelerate the large PDF catalog downloads for worldwide users, we introduce a CDN layer. Static files (PDF catalogs) reside in Azure Blob Storage, and we configure **Azure Front Door (or Azure CDN)** in front of the storage account. Azure Front Door is a global content delivery network that caches and delivers content from the nearest PoP. As Microsoft notes, “Azure Front Door enhances the delivery of static content from Azure Storage blobs, providing a secure and scalable architecture”. With Front Door/CDN, when a student in Europe or North America requests a catalog PDF, it is served from a nearby edge cache rather than the origin in Azure, drastically improving download speed and reducing load on the origin storage. We also enable HTTPS and, if needed, Web Application Firewall (WAF) on Front Door for security. This CDN integration directly addresses the slow download issue by leveraging global caching.

Identity Solution with Microsoft Entra ID

For authentication, we adopt Azure Active Directory (Entra ID) as the central identity provider. Entra ID is “a cloud-based identity and access management service” that provides a single identity control plane. We will synchronize the on-premises ADDS with Entra ID using Azure AD Connect (now called Entra ID Connect), so that student users keep their same credentials in the cloud. Application authentication will use Entra ID; App Service has built-in integration for Entra ID login. For external IT vendors, we leverage Azure AD B2B collaboration: vendors can be invited as guest users in our Entra ID tenant, allowing them to log in with their own credentials. Role-based access controls in Entra ID ensure that only authorized students or partners can

access each app. In summary, Entra ID provides secure, internet-accessible authentication for all users and supports both internal (student) and external (vendor) identities



Microsoft Azure Architecture and Migration guides are used for best practices. For example, Azure App Service and Azure Database are recommended for web and relational workloads, Azure Functions for event-driven tasks, Azure DevTest Labs for dev/test environments, and Azure Front Door/CDN for global content delivery. Cost optimization guidance (right-sizing, Reserved Instances, Hybrid Benefit) is followed. Each choice aligns with Azure's well-architected design principles and CUE's requirements (scalability, isolation, global performance, and identity). The diagrams above illustrate the proposed Azure architectures in block form with appropriate service icons.